Project ID	Project Title	Description
0	Lightweight DDoS / Port-Scan Detector using AF_XDP	**Description:** In this project you will learn and work with AF_XDP. Essentially, you will build and attach an AF_XDP program to a local or Pi5 network interface to inspect packets at (near) line rate, detect SYN-flood and port-scan patterns using simple rate-based heuristics, and log or drop offending flows. Compare against a naive userspace libpcap/Scapy-based detector for latency and CPU overhead. **Expected outcomes**: Understand the AF_XDP socket API and the kernel/userspace packet path. Implement a rate-based anomaly detector (e.g., SYNs/sec per source IP, distinct-port-per-source counting). Compare in-kernel (AF_XDP/XDP) detection against a userspace equivalent. Characterize detection latency and false-positive rate under a real attack simulation.
1	A Minimal VPN Tunnel	**Description**: Implement a simplified secure tunnel from first principles: ECDH key exchange, symmetric encryption (e.g., ChaCha20-Poly1305 via libsodium), and a TUN/TAP-based packet tunnel between two Pi5s over an untrusted WiFi link. Benchmark the throughput and CPU cost of encryption versus running the link in plaintext. **Expected Outcome**: Understand TUN/TAP interfaces and how a VPN tunnel intercepts and re-injects packets. Implement a secure key exchange (ECDH) and authenticated encryption for tunneled traffic. Demonstrate confidentiality (packet capture shows only ciphertext on the wire). Quantify the throughput/CPU overhead introduced by encryption. Note: You should have a working point-to-point encrypted tunnel carrying arbitrary IP traffic. Must have Wireshark capture demonstrating plaintext is never visible on the wire. Analyze Throughput and CPU-usage comparison: tunnel vs. plaintext WiFi link. **Tools**: TUN/TAP interfaces (Linux), libsodium or equivalent crypto library, UDP as the outer transport. Wireshark,
2	Ultra Fast Packet Crafter/Ping utility: Blitzping.	**Description:** This project ports Blitzping to DPDK for high-speed packet processing and adds hop-by-hop network tracing functionality. You need modify the original code to use DPDK's direct memory access, implement TTL-based traceroute with precise timing measurements, and build a simple GUI to display network paths in real-time. The enhanced tool will demonstrate performance improvements over standard networking utilities through hardware-accelerated packet handling. **Tools:** DPDK, Intel PMD drivers, Hugepages, NUMA libraries,Mininet, Raw sockets, Libpcap, blitzping, iperf3, tcpdump, GCC/Clang. **Expected Outcomes:** Learn how to speed up packet processing using DPDK by building a high-performance network system. Explore network behavior by implementing hop-by-hop tracing with precise timing, and compare the performance of DPDK-based tools with traditional networking applications. **Reference:** https://doi.org/10.1016/j.comcom.2020.07.040 Extend the opensource blitzping to provide custom functionalities: 1. Port blitzping/hping3 to run on DPDK. 2. Build hop-by-hop node and delay information with minimal GUI https://github.com/Thraetaona/Blitzping
3	Network Telemetry Framework	**Project Description:** Develop a standardized framework to collect, transport, and visualize network performance metrics such as bandwidth usage and latency from routers and switches in real time. The system gathers metrics at regular intervals, streams them to a central backend, and processes the data for visualization and to monitor network load, identify latency issues, and quickly respond to performance problems. **Tools/Technologies:** Python, gRPC, SNMP, REST APIs, OpenTelemetry, Grafana, or other telemetry protocols; reference: [RFC 9232]. **Expected Outcomes:** Real-time monitoring dashboards that display live bandwidth utilization and latency trends across devices. Threshold-based alerting for performance issues (e.g., high bandwidth usage or latency spikes). Historical data access and export through REST APIs for reporting and integration. A modular and scalable system that can be extended to support more devices or metrics without affecting network performance.
4	Per-Process Bandwidth Tracker using eBPF	**Description**: Build a real-time network usage tracker using eBPF to measure send/receive bandwidth per process. Support per-remote IP tracking, protocol filtering (TCP/UDP), and historical usage storage for reporting. **Tools** Linux eBPF (bcc / libbpf) Go or Python (user-space program) SQLite / Prometheus (storage) CLI or simple Web UI **Expected Outcomes** Hands-on eBPF networking experience Real-time per-process + per-IP bandwidth monitoring Protocol-based filtering (TCP/UDP) Historical usage reports

Project ID	Project Title	Description
5	Intelligent Network Configuration Assistant using MCP and Rule-Based Automation	**Project Description:** This project builds an intelligent assistant using MCP that helps network administrators configure devices securely and efficiently. It interprets user commands and applies configurations like firewall rules and bandwidth limits based on predefined policies and network context. **Tools/Technologies:** MCP frameworks (FastMCP, Anthropic MCP SDK), network tools (`iptables`, `tc`), scripting (Python, Bash), configuration files (YAML/JSON), monitoring tools (`ping`, `iperf3`), Docker for deployment. **Expected Outcomes:** Apply network configurations from user commands Validate changes through monitoring tools Provide structured, rule-based automation for secure and optimized settings **Deliverables**: Working assistant with example commands, rule files, deployment scripts, test cases, and a final report explaining setup, use cases, and validation results.
6	Multi-Path QUIC (MP-QUIC) vs MPTCP: Performance and Reliability Evaluation	**Project Description:** Study and compare MP-QUIC and MPTCP protocols for applications like video streaming, file transfer, and VoIP, focusing on throughput, latency, and connection stability. **Tools/Technologies:** QUIC implementation: quic-go, Cloudflare's quiche MPTCP: Linux kernel MPTCP, Multipath-enabled iproute2 Network emulation: NetEm, Mininet Monitoring: iperf3, Wireshark **Expected Outcomes:** Measure throughput, latency, and reliability under different network conditions Analyze how each protocol handles packet loss and congestion Gain practical experience configuring multipath protocols Provide recommendations for protocol use based on application needs
7	Extending Kubernetes Self-Healing for Networking Failure	**Project Description:**Kubernetes provides built-in self-healing features like restarting failed pods or rescheduling workloads when nodes go down. However, it does not automatically recover from networking-related failures such as CNI plugin crashes, CoreDNS issues, or broken NetworkPolicies. In this project, a custom Kubernetes operator/controller will be developed to detect and fix such failures. The operator will monitor: CNI plugin health (Calico/Flannel pods) DNS health (CoreDNS latency and failures) Pod-to-pod connectivity (via periodic probes) NetworkPolicy enforcement (detect unreachable services) When a problem is detected, the operator will apply automated remediation e.g., restarting pods, reapplying NetworkPolicies, or switching to a backup DNS configuration.. **Tools**: Kubernetes (Minikube/KIND) Prometheus for health metrics + Alertmanager Python (client-go) or Go (Kubebuilder) for operator development Calico/Flannel as CNI plugin Loki/Grafana for log monitoring (optional) **Expected Learning Outcomes:** Understand limits of Kubernetes' built-in self-healing Learn how to extend Kubernetes with custom controllers Debug and recover networking failures in clusters Gain hands-on experience with Kubernetes monitoring and networking stack

Project ID	Project Title	Description
8	DNS Resolver using AF_XDP	**Project Description**: This project implements a DNS resolver that sends and receives DNS packets using an AF_XDP (XSK) socket instead of the usual UDP/TCP socket. An XDP program attached to the NIC redirects DNS traffic (UDP dst port 53) to an AF_XDP socket; a user-space resolver crafts raw Ethernet/IPv4/UDP/DNS frames, transmits them via the XSK TX ring, and receives responses on the XSK RX ring. The project demonstrates low-latency, high-throughput packet I/O, zero-copy packet handling tradeoffs, and the extra plumbing required (UMEM, rings, descriptor management) compared to conventional sockets. **Tools / Technologies** Linux kernel AF_XDP / XDP support (recent kernel) clang/llvm (for compiling the XDP BPF program) libbpf / libxsk (or equivalent XSK helper code) for user-space XSK setup C (user-space resolver) for packet crafting and XSK ring handling bpftool / xdp-loader (to load/attach the XDP program) tcpdump / Wireshark / pkt-gen (for validation and packet inspection) **Expected Outcome:** Comparison vs standard UDP sockets: measurements of latency, throughput, and CPU overhead showing AF_XDP tradeoffs (zero-copy performance benefits vs implementation complexity).
9	Digital Twins for Large Scale Data Center Networks	Project Description: In this project, you will explore and simulate the networking infrastructure of a data center using NVIDIA Air (Digital Twin). You will create a digital twin model of a data center network, focusing on key components such as routers, switches, servers, and links. The goal is to simulate and visualize network traffic, latency, bandwidth usage, and failure scenarios to optimize and predict network behavior in the data center. You can extend and write python code to simulate and build any networking scenario like, use of machine learning to predict network traffic patterns and automatically adjust configurations based on predicted loads or simulate security aspects like DDoS attacks, firewall configurations, or encrypted traffic analysis into the network model. Tools / Technologies: NVIDIA Air Access (Register for a Developer Account) Python Expected Outcome: Learn about DigitalTwins and develop a fully functional DigitalTwin Model for network simulations. Demonstrate the Digital Twin and insights on traffic monitoring, prediction or security analysis.
10	SoNIC Application for Traffic Engineering	**Project Description**: Software for Open Networking in the Cloud (SONiC) is an open source network operating system (NOS) based on Linux that runs on switches from multiple vendors and ASICs. In this project, you will work on the SoNIC NOS and build a traffic monitoring system using the SDN controller that monitors network load and adjusts the paths taken by data packets to optimize for latency, bandwidth, or fault tolerance. **Tools/Technologies**: SoNiC, OpenFlow, ONOS Controller, Packet/Flow Generators like TRex. **Expected Outcome**: Demonstrate a dynamic network where traffic is continuously monitored and optimized to avoid congestion and maximize throughput. [optional] You can use the programmable switch to capture network traffic statistics (packet counts, bandwidth utilization, etc.), and build a system that analyzes traffic patterns and alerts administrators when unusual traffic behavior or potential security issues (e.g., DDoS attacks, abnormal traffic spikes) are detected.
11	TLS Fingerprinting	**Project Description**: Build a tool that passively captures TLS ClientHello and ServerHello messages and computes JA3/JA3S fingerprints (with JA4/JA4S as a stretch goal) to identify the client or server application/library generating the traffic. Curate a small reference database and demonstrate distinguishing real clients (browsers, curl, custom scripts) purely from their handshake fingerprint. **Tools/Technologies**: : libpcap/Scapy (or raw sockets) for capture; fingerprint computation implemented per the JA3/JA4 specifications, eBPF/XDP, Packet/Flow Generators like TRex, KV-stores (Redis, Valkey). **Expected Outcome**: Demonstrate the working JA3/JA3S fingerprint extractor, validated against published reference JA3 hashes for known clients; A curated database correctly identifying at least 5 distinct clients/tools by fingerprint alone. Demonstrate distinguishing, e.g., curl vs. a browser vs. a custom TLS client on the wire. Understanding on the fingerprinting's role in security monitoring (malware C2 detection, client identification) and its limitations, including fingerprint randomization in modern browsers as an evasion technique.
12	Comparative Analysis of QUIC/MASQUE VPN Tunnels	**Project Description**: Investigate how different implementations of QUIC/MASQUE-based tunneling behave under realistic network conditions and will compare their performance, functionality, and implementation characteristics. Select two or more open-source implementations of MASQUE for example, masque-go, masque-rs, masque-tunnel, Usque, or other mature MASQUE implementations—and construct a common experimental testbed so that the implementations are evaluated under identical conditions.The evaluation should compare CONNECT-UDP and, where supported, CONNECT-IP in terms of throughput, RTT/latency, packet loss, jitter, connection establishment time, CPU utilization, memory consumption, and tunnel overhead. At the end, you should provide a quantitative comparison and a recommendation of which implementation is most appropriate for particular scenarios such as low-latency applications, lossy networks, full-tunnel VPNs, or high-throughput UDP traffic. **Tools/Technologies**: : Go/Rust/C++, QUIC/HTTP/3/MASQUE, Linux network namespaces, tc/netem, iperf. **Expected Outcome**: Develop a reproducible benchmarking framework capable of testing multiple MASQUE/QUIC implementations under identical network conditions. Evaluation presented as quantitative graphs and tables comparing throughput, latency, jitter, packet loss, resource utilization, connection setup, and protocol overhead for different implementations and network scenarios. Present the observed differences in terms of QUIC congestion control, HTTP/3 multiplexing, QUIC DATAGRAMs, implementation architecture, and operating-system networking. **Reference**: https://dl.acm.org/doi/pdf/10.1145/3488660.3493806 (old paper)

Project ID	Project Title	Description
13	Scalable Network I/O: From *selec*	**Project Description**: Build and compare single-threaded TCP echo servers using four distinct Linux socket-handling mechanisms: select, poll, epoll, and io_uring. Implement each server paradigm from scratch and explore the shift from synchronous notification to truly asynchronous completion queue processing pattern. By using appropriate load generators (tcpkali, wrk etc.), students should benchmark and profile the behavior of these mechanisms by varying the connection loads (10, 100…) to observe system call overhead, memory scalability, and CPU efficiency. **Tools/Technologies**: C/C++/Rust, Linux network stack, io_uring **Expected Outcomes**: Upon completion, students will deliver fully functional C/C++ implementations for all four engines alongside a benchmarking report analyzing throughput, latency distributions, and system call frequencies. Students will demonstrate a clear understanding of O(N) vs O(1) scaling bottlenecks, explain why select and poll degrade under high concurrency, and identify the trade-offs that make io_uring's ring-buffer design superior to epoll for minimizing user-to-kernel context switches.